InstruMate: A Systematic Framework for Assessing Android App Repackaging Resilience

Leandro de Souza Oliveira*, Rodrigo Bonifácio*†, Joanna C. S. Santos‡, Rui Rua§
*Computer Science Department, University of Brasília, Brazil
†Informatics Center, Federal University of Pernambuco, Brazil
‡Department of Computer Science and Engineering, University of Notre Dame, USA
§NYU Abu Dhabi, UAE

***Abstract*—Repackaging is a widely used technique for modifying existing Android applications by introducing targeted changes to their code and resources. Malicious actors exploit it to distribute malware, commit plagiarism, and exfiltrate sensitive data. Although often described as a straightforward process, requiring only that an app be downloaded, decompiled, modified, and redistributed, its practical use in research contexts reveals a more complex reality. Existing work frequently overlooks why certain repackaging attempts fail, as well as the potential impact of repackage-proofing mechanisms embedded within apps. To address this gap, we present InstruMate, a novel approach for systematically assessing the repackaging resilience of Android applications. InstruMate progressively escalates from simple modifications—such as altering the app's signature—to more complex interventions targeting code instrumentation. Each modified version is rigorously assessed against the original using novel health-check procedures that reveal discrepancies in the user interface and execution behavior. We demonstrate this approach, by evaluating the resilience to repackaging of 156 highly popular apps obtained from the Google Play Store (including WhatsApp Messenger, LinkedIn, Roblox, TikTok, among others), each exceeding one billion installations worldwide. The results reveal that 86% of the analyzed apps are *non-resilient* to the most basic form of repackaging involving only a signature change; 81% can be modified to enable execution in *debug mode*; and 83% exhibit no resistance to superficial *appearance modifications*. Beyond these surface-level changes, more sophisticated transformations using black-box static and dynamic instrumentation show that 65% of the apps remain susceptible to code tampering. In contrast, 11% of the apps in our dataset actively resist repackaging through explicit user warnings or runtime exceptions that abruptly terminate execution. We further trace several of these defenses to specific anti-repackaging mechanisms integrated within the analyzed applications.**

***Index Terms*—Repackaging Android Applications, Android Application Security, Software Analysis and Manipulation, Empirical Methods in Software Engineering.**

I. Introduction

Android repackaging is a known technique for modifying existing Android applications (or "apps," for short), typically introducing specific changes to their code or resources and then redistributing the app in official or unofficial stores. Malicious actors exploit repackaging to spread malware, commit plagiarism, and exfiltrate sensitive information within the Android ecosystem [1]–[9]. Also, the possibility that repackaged apps may reach official marketplaces is particularly noteworthy, given that a broad user base relies on these markets as trusted repositories for software distribution. For example, Kalinin et. al. [10] reported the discovery of spyware embedded within repackaged variants of the popular messaging app WhatsApp. In an effort to improve the security of the official marketplace, Google has recently announced plans to implement a developer certification process [11].

Although Android app repackaging is commonly described as a straightforward process [12], requiring only that an app be downloaded, decompiled, modified, and redistributed, its practical application in research contexts reveals a more complex reality. Beyond its misuse by malicious actors, repackaging is frequently used by security researchers for tasks such as bug analysis, behavioral tracking, and data leak detection [13]–[20]. However, reported success rates vary widely and repackaging often proves more challenging than it appears (see Section II-C). These studies rarely explain *why* certain repackaged apps fail, citing only broad factors like process complexity [13] while overlooking potential repackage-proofing mechanisms [9]. Current approaches assess repackaging success by launching variants and monitoring execution for a predefined period [13]–[15], [19], [20]: no crash equals success. This evaluation is overly simplistic, ignoring anti-repackaging mechanisms and failing to detect behavioral changes from repackaging. For instance, such simplistic checks would fail to identify apps that successfully launch but employ more subtle defensive mechanisms, including explicit warning messages (e.g., Microsoft 365: *"This app may not be from a trusted source"*; CamScanner: *"This copy is not genuine"*), prompts to reinstall the app (e.g., YouCam Makeup app), delayed boot-loop behaviors (e.g., µTorrent via Google Dynamite), or behavioral changes such as newly requested permissions (e.g., Billiards Mate app).

To provide insight into this topic, we present InstruMate, *a framework for assessing whether an Android app is resilient to repackaging*, based on the creation of *variants*—i.e., controlled repackaged versions of an app. Each variant includes modifications to sensitive artifacts, simulating various forms of app modifications, including code tampering achieved through static and dynamic code instrumentation. The original app, along with its generated variants, undergoes a series of health check procedures, with the original app serving as the baseline to evaluate whether the app is susceptible to repackaging.

We assessed the capabilities of InstruMate through an empirical study using a dataset of 156 popular Android apps

available on the Google Play Store—including apps developed by Meta, Microsoft, and TikTok. Several of these apps have surpassed one billion installations worldwide. Our experiments revealed that 86% of the analyzed apps were not resilient to even the most basic forms of repackaging, such as merely altering the app's signature or making superficial appearance changes (e.g., renaming the app on the launcher screen). Even these basic modifications pose significant risks, as they can be exploited to redirect user traffic, exfiltrate sensitive data, or undermine developers' monetization strategies [6], [21], [22]. Beyond these "trivial" changes, INSTRUMATE's more advanced transformations—based on black-box static and dynamic instrumentation techniques—revealed that 65% of the apps were susceptible to code-level modifications.

In summary, this paper makes the following contributions:

- **INSTRUMATE:** a new method and tooling for progressively assessing the resilience of Android apps to repackaging.
- **Enhanced health-check procedures:** a systematic mechanism for verifying the success of repackaging attempts and identifying potential resilience gaps.
- **Empirical study:** an investigation showing that (a) many popular Android apps are not resilient to either simple resource modifications or advanced code tampering techniques, and (b) several recurring protection strategies are currently employed to prevent repackaging.

II. BACKGROUND AND RELATED WORK

This section outlines the key concepts of this paper, including the threat model of Android app repackaging and defense mechanisms against such attacks. The section concludes by positioning our research with respect to the related work.

A. Android Apps and Repackaging

Android apps are commonly delivered to users as an ***Android Package Kit (APK)*** file. These APK files may include precompiled Dalvik Executable (DEX) bytecode, resources (e.g., images, strings, layouts, etc.), precompiled code for different CPU architectures, and a manifest file containing meta-data information such as the app's signature, version, permissions, entry points, and components.

Although single APK files are still used for app distribution, they are no longer the standard file format used by the Google Play Store [23]. Since 2021, ***Android App Bundles*** (AAB) have been the standard format adopted by the official store. Unlike traditional APKs, an AAB contains the code and resources for multiple device configurations but defers the APK generation to Google [24], [25]. This format introduced split apps, which consist of a set of smaller, device-specific packages rather than a single monolithic APK. For example, at the time of this research, TikTok consisted of 14 split APKs.

Desnos and Gueguen [12] discussed ***Android app repackaging*** [26] by emphasizing the ease with which an app can be downloaded, decompiled, and repackaged before being redistributed, highlighting the need to develop methods to detect app similarities. Repackaging is a threat to the Android environment, where malicious actors use it to reconfigure advertisement frameworks, affecting the revenue of the original app [21] or to spread malware, demanding that market maintainers invest in powerful tools to limit the spread of repackaged apps [6], [22]. Unfortunately, app markets have grown so large that relying on app-to-app comparisons to search for repackaged apps is not feasible, as this approach results in a *combinatorial explosion*, making it impractical in real-world scenarios [22]. Figure 1 illustrates the repackaging threat model. Initially, developers publish legitimate apps to official and third-party marketplaces, including the Google Play Store (Step 1). End users typically access these marketplaces to download authentic versions of the apps (Step 2). However, this standard distribution pathway can also be exploited by malicious actors (Step 3), who apply repackaging techniques to modify them (Step 4), and subsequently redistribute the tampered versions through the same channels or unofficial channels (Step 5).

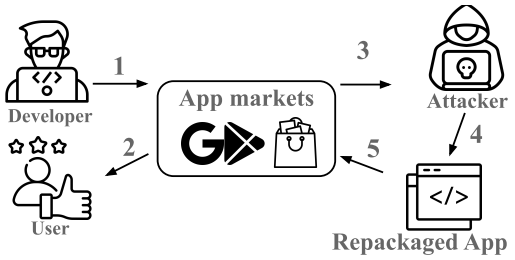


Fig. 1: Repackaging threat model and redistribution.

B. Defenses Against Repackaging

Obfuscation is a technique employed by legitimate developers to increase the difficulty of reverse engineering by making the code harder to interpret [27]. ***Watermarking***, on the other hand, serves to distinguish legitimate ownership by embedding identifiable markers within the app [28] that should survive the repackaging process. Although these techniques can discourage attackers, they do not provide a failsafe solution, as if these security mechanisms are bypassed, they cannot prevent repackaged apps from being distributed [9]. In contrast, ***repackage-proofing*** offers a more proactive approach by embedding defense mechanisms directly into the app [9]. This strategy detects repackaging when it occurs by introducing intentional code designed to trigger failures exclusively in repackaged variants, without affecting the original app's functionality. Repackage-proofing schemes often employ multiple strategically embedded guards within the app. When any of these guards are activated, they communicate the tampered status and induce crashes at locations distant from the detection site [29]. This approach not only complicates the attackers' task of identifying and disabling the mechanisms but also **creates an illusion of unpredictable behavior**. Examples of repackage-proofing schemes include *Droidmarking* [30], *SSN-Stochastic Stealthy Network* [9], and *AppWarder* [26].

C. Related Work

Black-box app assessment techniques, such as Mining Sandbox [16], Joe Sandbox Mobile [31], and RV-Android [20], typically rely on underlying instrumentation infrastructures [13]–[15], [19], [32] to systematically explore and extract behavioral information from Android apps.

Some tools rely on **static instrumentation**, which operates by manipulating an intermediate representation of the code. For example, COSMO's black-box approach [14] transforms Dalvik bytecode into Java bytecode and applies JaCoCo's offline instrumentation to inject additional instructions that record code execution. ACVTool [15] instruments Dalvik bytecode directly in its Smali representation, whereas DroidFax [19] and AndroLog [13] translate Dalvik bytecode into the Jimple intermediate representation [33] using the Soot framework [34] and then inject coverage-tracking statements.

Other tools modify the code at runtime through a process known as **dynamic instrumentation**. In this approach, specific instructions can be inserted, arguments and return values can be replaced, or entire functions can be modified, enabling changes to the program's original logic [32]. Frida [1] is a widely used dynamic instrumentation framework, and repackaging is commonly used to embed it directly into an app for redistribution. In this mode, Frida can either inject predefined instrumentation logic at startup or dynamically retrieve instrumentation scripts from an external server [32], [35].

These tools exhibit **varying success rates** when instrumenting Android apps [15]. For example, Samhi et al. [13] report that ACVTool successfully instrumented 48% of the evaluated apps, COSMO [14] 79%, and AndroLog 98%. In a large-scale experiment, Bao et al. [17] used DroidFax [19] and found that only 844 out of 2,750 apps (31%) could be successfully instrumented.

Unlike previous work that evaluates individual instrumentation techniques, INSTRUMATE provides a unified framework for systematic resilience assessment across multiple techniques, with rigorous health-checking procedures. The design of this framework and the types of variants produced are presented in Section III.

III. INSTRUMATE

This section presents **INSTRUMATE**, a repackaging infrastructure for building controlled repackaged variants. The INSTRUMATE's approach separates the unpacking and repacking steps from instrumentation, allowing the use of specialized tools optimized for each phase. The suite of strategies used to generate variants is centralized into a component, which we call *Variant Maker*. Various issues may arise during the construction of a variant, and even if the build process completes without errors, the variant may encounter runtime problems that render it non-functional. INSTRUMATE implements a **two-stage variant creation pipeline** (Figure 2) that accepts an app as input, along with a specification of the desired features to be included in the variant. INSTRUMATE then outputs either a repackaged app that meets the specifications or an error report.

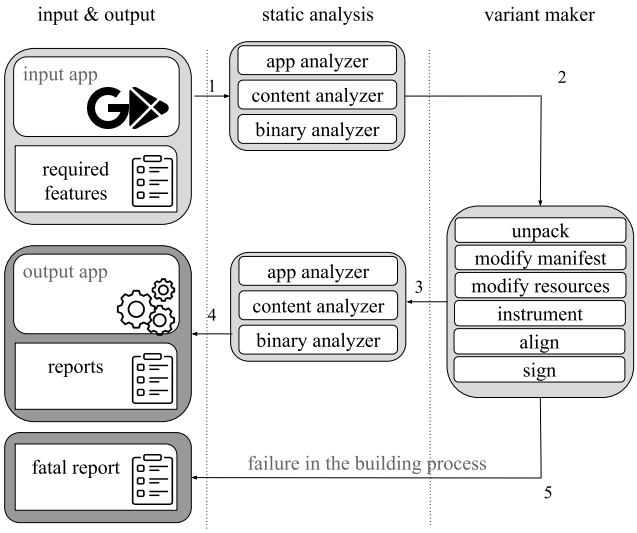


Fig. 2: INSTRUMATE's variant creation pipeline.

A. Static Analysis Stage

In the first stage, INSTRUMATE executes a set of static analysis tasks on a given app, involving an *app analyzer*, a *content analyzer*, and a *binary analyzer*. During the *app analyzer* task, INSTRUMATE leverages Androguard [12] to collect key app characteristics, including activity and service listings, permissions, package ID, version, signature information, and split count. Next, during the *content analyzer* task, INSTRUMATE computes the SHA-256 hash for each file in the app. In addition, it uses Apache Tika[2] to determine the MIME type of each app file. This process uniquely identifies each file by its hash and provides essential metadata for *variant makers*. Typically, Android apps store their compiled code within multiple DEX files included in the APK. During the *binary-analyzer* task, INSTRUMATE leverages Radare2[3] to analyze the binary files identified by Tika as: `application/x-dex`, `application/x-executable`, or `application/x-sharedlib`, and thus verify whether these files contain structures typically found in compiled code. The static analysis phase plays a crucial role in extracting information, as a detailed analysis of the package contents enables a more thorough comparison between the original app and its variants, particularly aiding in the identification of repackage failures. For this reason, **the same static analysis procedures are also performed on the resulting variant (Step 3 in Figure 2)**.

B. Variant Maker Stage

Variant makers are currently configured to generate repackaged variants with controlled modifications that preserve the

[1] https://frida.re/

[2] https://tika.apache.org/

[3] https://rada.re/

original program logic. These modifications are designed to exercise a range of app transformations, beginning with trivial alterations—such as modifying the app's digital signature, altering the manifest file, or changing resource elements like the app's name. The process then extends to more sophisticated transformations, including those introduced by static and dynamic instrumentation techniques. To facilitate analysis, the modifications were categorized into four groups: Signature Modifications, Manifest File Modifications, Resource Modifications, and Code Modifications.

(G1) Signature Modification*:* The Android platform requires that any update to an app must be signed with the same key as the version already installed. This ensures that updates originate from the same developer [15]. If someone alters and redistributes an app with a different signature, future updates must be signed with the same certificate. This can negatively impact the original app's revenue [21] and potentially introduce malware in future updates. Furthermore, merely changing the signature is a practice known as *Lazy Cloning* [21], which may be done for fun or fame [9]. It also serves as a prerequisite for more sophisticated modifications, such as those expected in the G2, G3, and G4 groups (detailed below). The proposed implementation generates variants in this group by replacing the signature of the original apps. The procedure for generating **G1 variants** consists of unpacking the APK archive, removing the original signature metadata, generating new signature files, and reassembling the modified package into a valid APK. To achieve this, two main variant makers were configured: ME, which uses ApkEditor; and MZ, which relies solely on standard ZIP operations. These variant makers operate on a per-APK basis, generating split variants for apps originally delivered as split APKs (see AAB in II-A). Additionally, the variant maker MEM was developed to merge split apps into a single APK. Since MZ lacks decoding capabilities, it was used exclusively to generate variants within the G1 group.

(G2) Manifest File Modifications*:* The manifest file is the primary configuration file for an Android app. By altering this file, an attacker can add, remove, or modify advertising IDs or request additional permissions [21]. Furthermore, modifying the manifest can enable the *debuggable* flag, allowing the use of profiling tools for inspection [36], [37]. The manifest file can also be configured to export sensitive data during backups or, in some cases, weaken the app's network security by crafting a vulnerable network configuration profile [38]. The proposed implementation generates variants in this group by changing the manifest file of an original app to enable debugging capabilities. If the *variant configuration specification* required enabling the app's debuggable mode, ME and MEM are then configured to decode the `AndroidManifest.xml` in the base APK, activate the debuggable flag, and generate outputs with this modification in addition to the signature change, thus producing split and merged variants within the G2 group.

(G3) Resource Modifications*:* Changes applied exclusively to the resource files can alter an app's name if it is stored in a resource string, modify the app's appearance, or adjust settings in configuration files, potentially impacting the app's behavior. According to [6], 91% of repackaged apps have modified resources compared to the original implementation. The generated variants in this group are configured to change the name of the app if its name is stored in a string resource item. Otherwise, it replaces a randomly selected string. To generate G3 group variants. INSTRUMATE also leverages the variant makers ME and MEM, configured to change the app's name referenced in string resources, and respectively produce split and merged variants.

(G4) Code Modifications*:* Code tampering can significantly impact an app's functionality, potentially introducing new features, altering existing behaviors, disabling security mechanisms, or exfiltrating sensitive user data. Such changes compromise the app's integrity and may lead to malicious outcomes. To simulate both static and dynamic code tampering, INSTRUMATE was configured to modify the original app's code using state-of-the-art static and dynamic instrumentation techniques. Static instrumentation was performed using three black-box coverage engines: Androlog [13], Aspect-based [20], and ACVTool [15]. Each of these engines operates at different abstraction levels: the Jimple intermediate representation, the Java bytecode, or directly at the SMALI code level. Dynamic instrumentation is achieved by injecting the Frida engine in embedded mode [39] with predefined instrumentation that simply confirms the injection at runtime. To produce the G4 variants, the variant makers MAND, MACV, and MASPE applied Androlog, ACVTool, and AspectJ-based instrumentation, respectively, all configured to capture method coverage. The variant maker MFRIE injected Frida in embedded mode. These variant makers operate on per split basis, while variant makers MANDM, MACVM, MASPEM, and MFRIEM applied these techniques to the merged variants. The instrumentation was carefully configured to avoid altering any of the existing program's variables, serving only to capture coverage, in the case of static instrumentation, or, in Frida's case, to inject into the process and gain access to the app's internal memory. Table I summarizes the current INSTRUMATE's variant makers. In this table, items with (*) indicate variant makers that leverage merging capability to produce a single merged APK. After creation, each variant undergoes a series of health check procedures, as detailed in Section III-C.

TABLE I: Variant makers per group. Items with (*) merge split apps to produce single APK variants.

Group	Variant Makers
G1	Me, Mz, MeM*
G2	Me, MeM*
G3	Me, MeM*
G4	Mand, Macv, MaspE, MfriE, MandM*, MacvM*, MaspEM*, MfriEM*

C. Health Check Procedures

As shown in Figure 3, for each app or variant, the health-check procedure installs the app, grants the required permis-

sions, and allows it to complete its initial setup. The app is then launched and monitored to record (i) **UI elements**, capturing the structural UI composition (interface components and static attributes such as *scrollable* and *clickable*, excluding dynamic state and text content), and (ii) **exception sites**, defined as tuples of the exception class and the corresponding launch point in the code. This enables straightforward comparisons across apps and variants.

This process can be repeated for an arbitrary number of iterations, with both the time window and the number of iterations defined as configurable parameters. INSTRUMATE relies on UI Automator to access the view representation and on Logcat to monitor stack traces. When applied to the original apps, the user-defined number of pipeline executions establishes a baseline for comparison. When applied to the variants, the same procedure assesses their health status by comparing the collected data against the baseline and subsequently assigning a classification, which may result in one of the following outcomes.

- **Healthy and compatible with the baseline**: This indicates that the variant does not exhibit differences in UI elements or observed exceptions when compared to the original app.
- **Healthy but incompatible with the baseline**: This indicates that the variant exhibits differences in terms of observed UI elements or thrown exceptions, but does not crash.
- **Faulty**: This indicates that the app failed to execute correctly, either by crashing, failing to launch, or failing to function properly.

It is important to note that this procedure includes several checks that have not been considered in previous research. Earlier studies considered a repackaged variant healthy if it could run without crashing for **three seconds** [15].

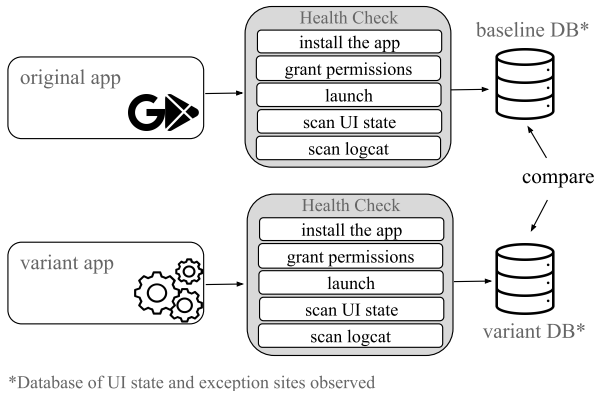


Fig. 3: Baseline construction and health check pipeline

D. Implementation Details

The current implementation of INSTRUMATE builds upon a set of established instrumentation and analysis tools, including AspectJ 1.9.6, ACVTool 2.3.2, ACVPatcher 1.0.0+a39ac, AndroLog v0.1, Frida 16.6.6, Apache Tika 2.9.2, PKEditor 1.4.1, ApkSigner 0.9, DEX Tools 2.4, and Radare2 6.0.4.

INSTRUMATE currently integrates the Google Monkey testing engine, although it is disabled by default. This design choice aligns with common practice in prior work reporting success rates for instrumentation techniques [13]–[15], [19], [20]. Stress testing may unintentionally expose bugs or trigger failures related to race conditions, network timeouts, or memory pressure—issues that can arise even in well-maintained apps and may inflate the number of faulty variants for reasons unrelated to the repackaging process.

For the experiments detailed in the following sections, we test each original app and its variants on a freshly wiped emulator instance, ensuring no prior usage. The emulator is configured as a Google Pixel 7 Pro AVD in portrait orientation, running Android 15 (API 35, x86_64), non-rooted, and provisioned with 20 GB of RAM and 20 GB of internal storage. Each original app is observed for one minute. In contrast, each variant undergoes a health-check procedure with an observation period of **two minutes**, corresponding to twice the duration used for the original app.

IV. EXPERIMENTAL DESIGN

The primary objective of the experiment we present here is to assess the capability of INSTRUMATE to determine whether Android apps can be successfully repackaged and to evaluate how effectively the proposed infrastructure identifies repackaging failures. To address this goal, the following research questions are formulated.

(RQ1) To what extent can INSTRUMATE determine whether Android apps are susceptible to repackaging?

(RQ2) Which repackaging defenses can be identified using the proposed framework?

(RQ3) What are the underlying factors that contribute to repackaging failures?

According to Khanmohammadi *et. al.* [3] and Li *et. al.* [6], popularity is the main criterion for an app being chosen for repackaging by malicious actors. That is why a subset of the most popular apps distributed primarily through the Google Play Store was selected, as detailed in Section IV-A.

A. Dataset of Selected Android Apps

We curated a dataset of Android apps by applying the following selection criteria to the set of 500 top-most downloaded apps reported by the Android Rank project [40]: (1) apps had to be free, stable, and installable by any user; (2) only apps with more than 50 million downloads were included; and (3) apps exhibiting sporadic crashes were excluded to ensure reliable attribution of failures in the repackaged variants. From the initial set, we excluded 136 apps (filter 1) and 208 apps (filter 3), mostly games, that failed to run on the emulator due to GPU limitations. Each remaining app was manually verified, and only fully functional apps were retained. All selected apps satisfied filter 2.

In total, from the top-most downloaded apps on the Google Play Store [40], we selected **156** stable, popular, free apps.

The dataset includes widely used applications such as WhatsApp, Facebook, Facebook Messenger, and Instagram, each exceeding 5 billion downloads worldwide. It also comprises less popular, yet still widely adopted apps, including eBay Online Shopping, Microsoft Teams, Mercado Libre Online Shopping, and CAIXA Tem (Finance), all with over 50 million downloads globally. The largest app in the dataset is the game *Asphalt Legends Unite* (3.2 GB), which includes 13 splits, followed by the 1.1 GB adventure game *Mini World CREATA*. TikTok comprises 14 splits.

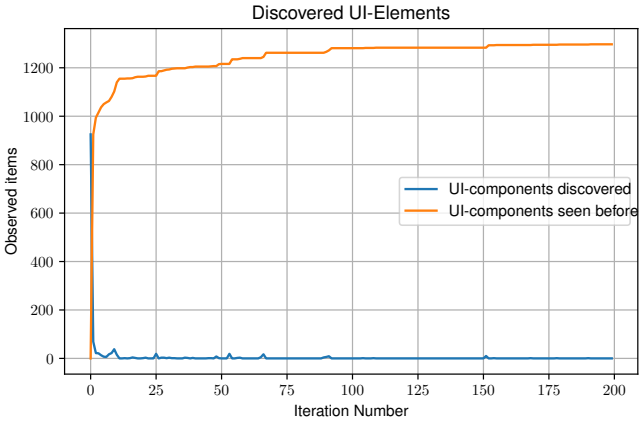
Table II characterizes the final dataset by market category, reporting the average app size (in megabytes), the number of splits, and the number of DEX files per category. Overall, each app consists of three splits: the base APK, a split containing architecture-dependent binaries, and a split containing graphical assets.

B. Baseline Construction

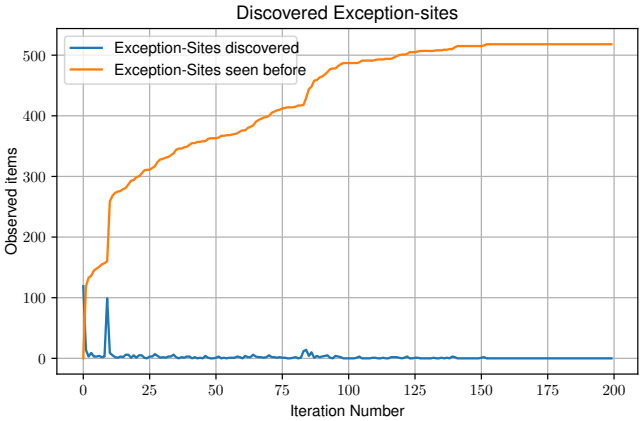
Each original app was monitored over 200 observations to construct the baseline database. Figure 4 illustrates this process. In the top chart, the plot illustrates the relationship between the number of new UI elements discovered and the total number of cumulative UI elements observed. Similarly, in the bottom chart, the plot shows the discovery of new exception sites and the cumulative number of exception sites previously observed. These figures demonstrate that the baseline established by observing the original apps successfully captured all of their UI elements and exceptions, thereby enabling the identification of previously unseen UI elements and exceptions introduced after the repackaging procedures.

TABLE II: Distribution of the number of apps (#Apps) in the dataset by category. All values for size, split count (#Split), DEX count (#DEX) are averages per category.

Category.	Size (MB)	#Split	#DEX	#Apps
Games	273	3.09	5.58	43
Productivity & Tools	111	3	8.45	33
Social & Communication	103	3.83	8.92	24
Media & Entertainment	101	3.5	7.6	20
Finance & Shopping	72	3	7.31	16
Travel & Navigation	117	2.7	9.8	10
Health & Lifestyle	89	3.4	7.8	5
Education	90	3.2	8.0	5



(a) UI-componentes saturation



(b) Exception-sites saturation

Fig. 4: Process used to construct the baseline database, which was formed by discovered UI elements and exception sites identified across increasing iterations.

V. Case Study Results

We executed the InstruMate on the 156 original apps, resulting in the successful generation of 1,693 variants, which corresponds to 72% of the expected total of 2,340. A significant portion of the creation failures was concentrated in the G4 group. Following their creation, each variant was subjected to the health check procedures. A detailed summary of both the creation results and the subsequent verification results is presented in Table III. Column **B** indicates the number of successfully built variants per variant maker, while column **H** represents the number of healthy variants. Similarly, column **F** denotes the faulty variants, and column **I** contains the number of healthy but incompatible variants. The data on the success rates of other variant makers in producing healthy repackaged apps in the same group serve as the basis for calculating the **HVG** column (**Healthy Variant Gap**). This column quantifies the number of healthy variants that a specific variant maker failed to produce while others in the same group succeeded.

A. (RQ1) To what extent can InstruMate determine whether Android apps are susceptible to repackaging?

We successfully built all variants using the **G1 group** of variant makers. Among these, Me produced 132 *healthy and compatible* variants, 13 *faulty* variants, and one *healthy but incompatible* variant, achieving a higher success rate compared to the variant makers Mz and MeM. Still focusing on the G1 group, out of the 156 apps, 134 (86%) were classified as **non-resilient** to signature modification. Note that this information is presented in the last row in the G1 group in Table III, and is derived by counting the number of apps for which at least one *healthy and compatible* variant was successfully generated.

TABLE III: Summary of health check results per variant maker. Columns: B (Built Variants), H (Healthy Compatible), F (Faulty), I (Healthy but Incompatible), HVG (Healthy Variant Gap). All percentage values are calculated with respect to the total of 156 apps.

Group	Variant Maker	B		H		F		I		HVG	
G1	Me - ApkEditor Unpack & Repack	156	100%	132	85%	21	13%	3	2%	2	1%
	Mz - Naive Zip Unpack & Repack	156	100%	117	75%	36	23%	3	2%	17	13%
	MeM - ApkEditor Unpack & Repack, Merged Variants	156	100%	129	83%	25	16%	2	1%	5	4%
	Me + Mz + MeM			134	86%						
G2	Me - ApkEditor Unpack & Repack	130	83%	120	77%	10	6%	0	0%	7	6%
	MeM - ApkEditor Unpack & Repack, Merged Variants	134	86%	127	81%	7	4%	0	0%	0	0%
	Me + MeM			127	81%						
G3	Me - ApkEditor Unpack & Repack	130	83%	125	80%	5	3%	0	0%	5	4%
	MeM - ApkEditor Unpack & Repack, Merged Variants	134	86%	129	83%	5	3%	0	0%	1	1%
	Me + MeM			130	83%						
G4	Mand - Instrumented with Androlog	78	50%	45	29%	30	19%	3	2%	57	56%
	Macv - Instrumented with ACVTool	111	71%	24	15%	85	54%	2	1%	78	76%
	MaspE - ApkEditor Unpack & Repack, Instrumented with AspectJ	65	42%	14	9%	43	28%	8	5%	88	86%
	MfriE - ApkEditor Unpack & Repack, Instrumented with Frida	126	81%	51	33%	74	47%	1	1%	51	50%
	MandM - ApkEditor Unpack & Repack, Merged Variants, Instrumented with Androlog	16	10%	9	6%	7	4%	0	0%	93	91%
	MacvM - ApkEditor Unpack & Repack, Merged Variants, Instrumented with ACVTool	106	68%	25	16%	81	52%	0	0%	77	75%
	MaspEM - ApkEditor Unpack & Repack, Merged Variants, Instrumented with AspectJ	65	42%	14	9%	44	28%	7	4%	88	86%
	MfriEM - ApkEditor Unpack & Repack, Merged Variants, Instrumented with Frida	130	83%	51	33%	79	51%	0	0%	51	50%
	Mand + Macv + … + MfriEM (all G4)			102	65%						

Since signature modification is a prerequisite for producing G2 to G4 variants, only these 134 apps marked as non-resilient in G1 were selected for further evaluation by the other groups.

With regard to Manifest File Modifications (**group G2**), the variant makers ME and MEM successfully built 130 and 134 variants, respectively. Among these, ME produced 120 *healthy and compatible* variants and 10 *faulty* ones, while MEM produced 127 *healthy and compatible* and seven *faulty* variants. MEM achieved a 95% success rate in producing *healthy and compatible* variants in group G2. No variant generated in group G2 was classified as *healthy but incompatible*. We observed similar results for **group G3**, where neither ME nor MEM produced any *healthy but incompatible* variants. ME and MEM achieved a 96% success rate in producing *healthy and compatible* variants in the group G3. Out of the initial 156 apps, 127 (81%) apps were classified as **non-resilient** to modifications in the G2 group and allowed execution in *debug mode*, and 130 apps (83%) were classified as **non-resilient** to modifications to resources stored in string files.

> **Finding 1.** The lower HVG observed for MEM in the G2 and G3 groups suggests that minor decoding issues were mitigated by the merging process.

Finally, the variant makers in **group G4** demonstrated the lowest overall success rates, accompanied by high variability. Within this group, MFRIE and MFRIEM achieved the highest numbers of *healthy and compatible* variants, each producing 51. They were followed by MAND, which generated 45 such variants. MASPE and MASPEM were the lowest ranked, each producing 14 *healthy and compatible* AspectJ-instrumented variants. No clear benefit was observed from the application of the merging process; conversely, its application in the case of Androlog resulted in decreased performance, with the number of *healthy and compatible* variants dropping to just nine. MACV and MACVM exhibited high construction rates; however, a substantial proportion of the repackaged variants they produced were faulty—77% and 76%, respectively. **On average**, within the G4 group, the success rate for merely generating repackaged variants was 65%, while the success rate for producing *healthy and compatible* variants dropped to **35%**. It is important to note that the health check procedures introduced an additional layer of rigor, imposing stricter validation criteria that further reduced the number of resulting *healthy and compatible* variants. Of the initial set of 156 apps, 102 (65%) were classified as **non-resilient** to code section modifications.

> **Finding 2.** Modifications applied before instrumentation were generally simpler, while code instrumentation often failed during the variant creation or generated non-launchable variants.

> **Finding 3.** No single instrumentation approach proved universally effective. The most successful variant makers—Frida-based (MFriEM) and Androlog-based(Mand)—

produced 33% and 29% of healthy variants, respectively. Combining multiple techniques increased the success rate to 65%, demonstrating that diversity maximized the number of successfully instrumented variants.

B. (RQ2) Which repackaging defenses can be identified using the proposed framework?

During the experiments with variant makers in Groups G1 and G4, the following repackaging defenses were observed.

1) Signature Modification (G1): Variants derived from two apps (Microsoft OneNote and Microsoft 365) displayed the message *"This app may not be from a trusted source"* and variants derived from one app (CamScanner) displayed the message *"This copy is not genuine"* (Figure 5). A manual inspection revealed a recurring pattern of exception sites originating from the same shared library (`libplat.so`) across five Android apps: Microsoft Excel, Microsoft 365, Microsoft OneNote, Microsoft PowerPoint, and Microsoft Word. Some variants of these apps displayed a modified user interface, while others simply closed upon launch. However, all variants exhibited exception sites that were traced back to `libplat.so`, indicating that this library functions as a platform abstraction layer used in multiple apps, and that it implements a mechanism to prevent code tampering.

Symbols in `libplat.so` (static analysis):
`CreateFileW, ReadFile, WriteFile, DeleteFileW, VirtualAlloc, HeapAlloc, CreateEventW, SleepEx, CreateMutexW, RegOpenKeyExW, RegQueryValueExW, GetSystemTimeAsFileTime, isDebugBuild, isTestBuild, isDevApk, getAppPackageName, getAppStoreName, CryptVerifySignatureW`

Other three non-healthy variants (apps: Carrom Pool Disc Game, Roblox and Beach Buggy Racing) produced IO exceptions[4] that were traced back to `libpairipcore.so`, which is publicly known to be a repackage-proofing solution[5]. One faulty variant (app: HP Printer Setup) reported no new exception site, but simply invoked `System.exit`, after loading the library `libfusg.so`. There is no public information about `libfusg.so`, however, InstruMate's static analysis found string symbols possibly related to repackaging defenses, and that could be revealed in certain circumstances: *"Stop poking around my nether bits you cretin!", "And you keep wondering why nobody likes you?"* and *"Go pound bits!"*.

2) Code Modifications (G4): Variants derived from one app (Game Pooking Billiards City) generated new UI elements displaying intimidating messages that prompt the user to grant the app permission to capture sensitive data (*"Allow Billards-mate to access all device logs"*) and variants derived from another app (MyJio) exposed an indication of repackaging detection, showing the message *"Rooted device detected"*, despite the fact that the emulator device was not rooted (Figure 6). Other variants derived from two apps (Score! Hero and YouCam Makeup) presented the messages *"Your device may not be compatible"* and *"Your installation is corrupted"* and prompted the user to reinstall the app. One exception site[6] appeared in three non-healthy variants (Myntra, µTorrent and VKontakte). This exception site was traced back to the **Google Dynamite library**, which forced the app to restart. In such cases, the Logcat message *"Module config changed, forcing restart due to module ads.dynamite"* was observed. Although public information about this proprietary library is limited and does not confirm whether the observed behavior is intentional, it effectively prevented the completion of the health check procedures on the generated variants.

Finding 4. No app exhibited defenses against repackaging within the G2 and G3 groups, indicating tolerance to execution in debug mode and to superficial modifications of resource files.

Finding 5. With regard to potential defenses that could induce sporadic failures—such as those implemented by SSN networks [9]—we did not observe variants that occasionally behaved consistently with the baseline while, at other times, crashed. This suggests that such complex defensive mechanisms were either absent from the apps in our dataset or not triggered under the experimental conditions.

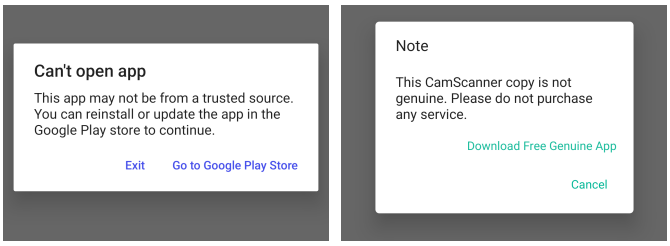


Fig. 5: Messages observed only in variants (group G1).

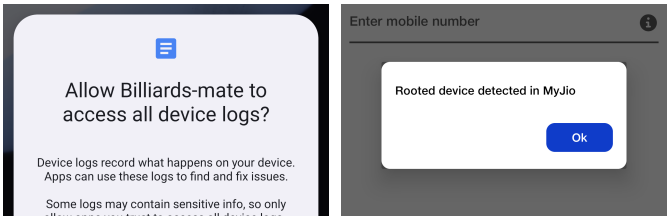


Fig. 6: Messages observed only in variants (group G4).

C. (RQ3) What are the underlying factors that contribute to repackaging failures?

In addition to repackage-proofing measures, we also identified other likely root causes of failures. As shown in Table

[4] `java.io.IOException` **at** `c.p.VMRunner.readByteCode`
[5] https://github.com/Solaree/pairipcore
[6] `java.io.IOException` at `m.ct.a`

III, MZ (the naive ZIP-based variant maker) participated in the generation of 117 healthy and compatible variants, but 134 were classified as non-resilient to signature modifications. To account for the increase to 134, other variant makers successfully generated healthy and compatible variants for apps that MZ could not produce. This gap is represented in the HVG column of Table III. Variant maker MZ failed to generate variants because certain files within the APKs are required to be stored uncompressed in the ZIP archive. A simple example is the shared object file, which has the `.so` extension, and is read directly from the APK and mapped to memory for execution. Analysis of MZ's HVG revealed IO exceptions during access of files directly stored in the APK, expecting them to be uncompressed and ready to be mapped to RAM memory at runtime, a technique which is particularly useful for large apps, such as games.

With respect to code instrumentation, some variant makers failed during the build process, and others generated APKs that could not be installed. While many of the issues require in-depth, app-specific investigation—beyond the scope of this experiment—some failures could be isolated. For instance, failures affecting variant creation for 33 apps were traced to potential issues such as concurrency bugs and misinterpretations of DEX opcodes in the Soot engine, which is used by Androlog. ACVTool failed to instrument certain methods due to register management issues, which triggered a Python `KeyError` in its register map. Additionally, it encountered a "list index out of range" error in `insn3rc.py`, suggesting the presence of a potential bug. Dex2jar failed to generate the expected JAR file from the provided DEX file for 26 apps, while Frida, in embedded mode, encountered issues when loading pre-existing shared objects in 10 apps.

Finding 6. We observed significant failures in instrumentation tools consistent with underlying bugs, affecting 69 apps.

VI. Discussion

In this section, we answer our research questions, present some implications of our research, and highlight some threats to the vaility of our study.

A. Answers to the Research Questions

Out of the 156 initial apps, 134 (86%) were found to be non-resilient to the simplest form of repackaging—merely changing the app's signature. Our analysis also revealed that 127 apps (81%) permitted execution in debug mode, enabling reverse engineering techniques that attackers could potentially exploit. Additionally, 130 apps (83%) offered no resistance to superficial modifications, such as changes to the app name. Furthermore, using black-box static and dynamic instrumentation techniques, it was confirmed that 102 apps (65%) were susceptible to modifications in their code sections. We also found recurrent patterns of defense mechanisms that work as repackage-proofing solutions. These findings provide critical insights toward answering our first and second research questions.

Answer to RQ1: The variant makers successfully repackaged 86% of the apps in our dataset, indicating that the analyzed apps exhibited a high degree of susceptibility to repackaging.

Answer to RQ2: A small subset of apps employed libraries such as `libplat.so`, which seems to be a platform abstraction layer used by multiple apps, and `libpairipcore.so`, a repackage-proofing library, to detect repackaging attempts and block modified variants. In addition, **Google Dynamite** ads library detected repackaging and induced boot loops in the apps. In contrast, no defenses were observed in the G2 and G4 groups, indicating tolerance to debug-mode execution and superficial resource modifications.

Besides defense mechanisms, the variant makers also failed to instrument several apps. We identified several factors that prevented variant makers from producing working variants, including the internal structure of APKs, potential bugs, and the misinterpretation of DEX opcodes.

Answer to RQ3: Variant makers in group **G4** struggle to build working variants due to the internal structure of the APKs, potential bugs in the tools, and misinterpretation of DEX opcodes.

B. Implications and Contributions

Methodological contribution: we introduce a rigorous health assessment procedure that addresses critical limitations in existing approaches. Previous research classified repackaged variants as faulty, if they failed, or healthy, if they launched and survived three seconds [13]. This criterion proves to be inadequate for modern Android apps. We propose a three-tier classification system that distinguishes between (1) healthy and compatible variants, (2) healthy but incompatible variants exhibiting functional differences from the original, and (3) faulty variants. This refinement enables researchers to identify subtle behavioral modifications that might otherwise go undetected.

Security perspective: only a small fraction of apps in our dataset implemented active repackaging defenses. This finding reveals a concerning gap in protection for millions of users who remain vulnerable to campaigns distributing repackaged, potentially malicious variants (note that WhatsApp, LinkedIn, Microsoft 365, Pinterest, Roblox, TikTok, Telegram, and CapCut Video Editor, all of which have surpassed one billion installations worldwide).

Progressive verification framework: our approach esca-

lates from signature replacement through debuggable flag activation and resource modification to code transformation, and provides a structured methodology for evaluating app susceptibility to repackage.

Ethical Considerations: our study revealed that several popular apps are susceptible to repackaging. In adherence to responsible disclosure practices, we opted to inform the respective developers about these findings and provided recommendations aimed at strengthening their apps against such threats, focusing particularly on the perspectives offered by the groups G1 to G4.

C. Threats to Validity

Our empirical assessment relies on several assumptions that could potentially threaten the validity of our results. First, to investigate stochastic security defenses such as those described in [9], we excluded several apps that crashed during execution. This procedure aimed to ensure that any stochastic behavior observed could be attributed specifically to the repackaging process, since the original versions of these apps executed deterministically. We acknowledge that this decision reduced the dataset size and thus may limit the *external validity* of our results—our findings may not generalize to all apps, especially those employing crash-inducing defenses. Nevertheless, this step improved *construct validity* by isolating the phenomenon of interest and avoiding confounding factors such as unrelated stability issues. We therefore deliberately traded generalizability for a more precise measurement of repackaging defenses. Under this controlled setting, no repackaged variants exhibited intermittent behavior attributable to such defenses (Finding 5).

Second, one might argue that the instrumentation techniques we employed are not exhaustive and that a realistic attacker would likely modify only small portions of the code. However, our choice to apply widespread instrumentation across all groups of variant makers was intentional: it ensured broad coverage and enabled us to verify whether any security defenses would be triggered under extensive code modifications. This setup increases our confidence that the absence of detected defenses is not due to limited code intervention or to the specific design of the implemented variant makers. Moreover, such an approach may also assist developers of repackaging defenses in identifying code regions that remain unprotected. We acknowledge, however, that this design choice may affect the *external validity* of our results, as findings may not generalize to other repackaging techniques involving lightweight or more targeted instrumentation. Finally, we do not claim completeness with respect to all possible repackaging procedures—achieving such completeness would be impractical in a systematic study of this nature.

While the previous discussion concerns the extent of our instrumentation, a related consideration involves the *choice of tools* used to implement it. For the variant makers in groups G1–G3, we employed widely adopted tools commonly used in Android repackaging research. For group G4, we explored both static and dynamic code instrumentation, leveraging state-of-the-art, actively maintained, and recently published instrumentation and code coverage tools. We argue that the use of these pre-integrated tools should not be viewed as a limitation but rather as a design choice, as they operate at distinct abstraction levels frequently examined in the literature [13]–[15], [41]. Furthermore, integrating additional tools — such as a variant maker based on ApkTool [42], a planned future addition — would likely increase the number of generated repackaged variants. This would not alter the results achieved so far; instead, expanding the toolset is expected to improve the accuracy and completeness of the results.

Although RQ3 identifies patterns in instrumentation failures, comprehensive root cause analysis would require controlled experiments isolating specific libraries or instrumentation techniques. InstruMate's infrastructure provides a foundation for such future investigations.

VII. Conclusion

In this paper, we introduced and evaluated InstruMate, a systematic approach for (a) generating repackaged versions of Android applications in a controlled manner and (b) assessing their resilience to repackaging. To produce repackaged variants, InstruMate integrates multiple variant makers that apply transformations ranging from signature modifications to static and dynamic code instrumentation. Using a curated dataset of 156 popular Android apps, InstruMate generated 1,693 repackaged variants, which we analyzed through a novel health-check procedure designed to reveal the extent to which apps resist repackaging attempts. Our findings show that 86% of the analyzed apps were susceptible to repackaging, from simple modifications to advanced code tampering. Also, InstruMate effectively detects common repackaging defenses by capturing traces observable on Android devices, relying only on ADB and UI Automator. These results suggest that InstruMate can support developers and researchers in systematically identifying weaknesses in existing protection mechanisms and incorporating effective anti-repackaging defenses early in the software lifecycle.

VIII. Data Availability

We release **InstruMate**'s source code and evaluation setup through the following link https://doi.org/10.5281/zenodo.17964704 and https://github.com/leosol/InstruMate.

Acknowledgments

This work was partially supported by INES.IA (National Institute of Science and Technology for Software Engineering Based on and for Artificial Intelligence) www.ines.org.br, CNPq grant 408817/2024-0, by the OLX Security Team, a global marketplace platform (www.olx.com.br), by the Brazilian Association of Forensic Examiners (ABPC – Associação Brasileira de Peritos em Criminalística, www.abpc-df.com.br), and by the Police of the Federal District (PCDF), Brazil.

REFERENCES

[1] F. Lyu, Y. Lin, J. Yang, and J. Zhou, "Suidroid: An efficient hardening-resilient approach to android app clone detection," in *2016 IEEE Trustcom/BigDataSE/ISPA*. IEEE, 2016, pp. 511–518.

[2] Y. Shen, P.-A. Vervier, and G. Stringhini, "A large-scale temporal measurement of android malicious apps: Persistence, migration, and lessons learned," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 1167–1184.

[3] K. Khanmohammadi, N. Ebrahimi, A. Hamou-Lhadj, and R. Khoury, "Empirical study of android repackaged applications," *Empirical Software Engineering*, vol. 24, pp. 3587–3629, 2019.

[4] Y. Zhou and X. Jiang, "Dissecting android malware: Characterization and evolution," in *2012 IEEE symposium on security and privacy*. IEEE, 2012, pp. 95–109.

[5] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon, "Androzoo: Collecting millions of android apps for the research community," in *Proceedings of the 13th international conference on mining software repositories*, 2016, pp. 468–471.

[6] L. Li, D. Li, T. F. Bissyandé, J. Klein, Y. Le Traon, D. Lo, and L. Cavallaro, "Understanding android app piggybacking," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 2017, pp. 359–361.

[7] J. Crussell, C. Gibler, and H. Chen, "Attack of the clones: Detecting cloned applications on android markets," in *Computer Security–ESORICS 2012: 17th European Symposium on Research in Computer Security, Pisa, Italy, September 10-12, 2012. Proceedings 17*. Springer, 2012, pp. 37–54.

[8] ——, "Scalable semantics-based detection of similar android applications," in *Proc. of ESORICS*, vol. 13. Citeseer, 2013.

[9] L. Luo, Y. Fu, D. Wu, S. Zhu, and P. Liu, "Repackage-proofing android apps," in *2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2016, pp. 550–561.

[10] D. Kalinin, "Analysis of a spy module inside a whatsapp mod," Nov 2023. [Online]. Available: https://securelist.com/spyware-whatsapp-mod/110984/

[11] R. Lakshmanan, Oct 2025. [Online]. Available: https://thehackernews.com/2025/08/google-to-verify-all-android-developers.html

[12] A. Desnos and G. Gueguen, "Android: From reversing to decompilation," *Proc. of Black Hat Abu Dhabi*, vol. 1, pp. 1–24, 2011.

[13] J. Samhi and A. Zeller, "Androlog: Android instrumentation and code coverage analysis," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 597–601.

[14] A. Romdhana, M. Ceccato, G. C. Georgiu, A. Merlo, and P. Tonella, "Cosmo: Code coverage made easier for android," in *2021 14th IEEE conference on software testing, verification and validation (ICST)*. IEEE, 2021, pp. 417–423.

[15] A. Pilgun, O. Gadyatskaya, Y. Zhauniarovich, S. Dashevskyi, A. Kushniarou, and S. Mauw, "Fine-grained code coverage measurement in automated black-box android testing," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–35, 2020.

[16] K. Jamrozik, P. von Styp-Rekowsky, and A. Zeller, "Mining sandboxes," in *Proceedings of the 38th International Conference on Software Engineering*, 2016, pp. 37–48.

[17] L. Bao, T.-D. B. Le, and D. Lo, "Mining sandboxes: Are we there yet?" in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2018, pp. 445–455.

[18] F. H. da Costa, I. Medeiros, P. Costa, T. Menezes, M. Vinícius, R. Bonifácio, and E. D. Canedo, "Droidxp: A benchmark for supporting the research on mining android sandboxes," in *2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2020, pp. 143–148.

[19] H. Cai and B. G. Ryder, "Droidfax: A toolkit for systematic characterization of android applications," in *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2017, pp. 643–647.

[20] P. Daian, Y. Falcone, P. Meredith, T. F. Şerbănuţă, S. Shiriashi, A. Iwai, and G. Rosu, "Rv-android: Efficient parametric android runtime verification, a brief tutorial," in *Runtime Verification: 6th International Conference, RV 2015, Vienna, Austria, September 22-25, 2015. Proceedings*. Springer, 2015, pp. 342–357.

[21] C. Gibler, R. Stevens, J. Crussell, H. Chen, H. Zang, and H. Choi, "Adrob: Examining the landscape and impact of android application plagiarism," in *Proceeding of the 11th annual international conference on Mobile systems, applications, and services*, 2013, pp. 431–444.

[22] L. Li, T. F. Bissyandé, and J. Klein, "Rebooting research on detecting repackaged android apps: Literature review and benchmark," *IEEE Transactions on Software Engineering*, vol. 47, no. 4, pp. 676–693, 2019.

[23] Anonymous, "About Android App Bundles — Android Developers — developer.android.com," https://developer.android.com/guide/app-bundle, 2021, [Accessed 08-06-2024].

[24] H. Bagheri, J. Wang, J. Aerts, N. Ghorbani, and S. Malek, "Flair: efficient analysis of android inter-component vulnerabilities in response to incremental changes," *Empirical Software Engineering*, vol. 26, pp. 1–37, 2021.

[25] Y. Liu, Y. Ma, X. Xiao, T. Xie, and X. Liu, "Legodroid: flexible android app decomposition and instant installation," *Science China Information Sciences*, vol. 66, no. 4, p. 142103, 2023.

[26] H. Ma, S. Li, D. Gao, and C. Jia, "Secure repackage-proofing framework for android apps using collatz conjecture," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3271–3285, 2021.

[27] M. Protsenko, S. Kreuter, and T. Müller, "Dynamic self-protection and tamperproofing for android apps using native code," in *2015 10th International Conference on Availability, Reliability and Security*. IEEE, 2015, pp. 129–138.

[28] C. S. Collberg and C. Thomborson, "Watermarking, tamper-proofing, and obfuscation-tools for software protection," *IEEE Transactions on software engineering*, vol. 28, no. 8, pp. 735–746, 2002.

[29] H. Ma, S. Li, D. Gao, D. Wu, Q. Jia, and C. Jia, "Active warden attack: On the (in) effectiveness of android app repackage-proofing," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3508–3520, 2021.

[30] C. Ren, K. Chen, and P. Liu, "Droidmarking: resilient software watermarking for impeding android application repackaging," in *Proceedings of the 29th ACM/IEEE international conference on Automated software engineering*, 2014, pp. 635–646.

[31] S. Neuner, V. Van der Veen, M. Lindorfer, M. Huber, G. Merzdovnik, M. Mulazzani, and E. Weippl, "Enter sandbox: Android sandbox comparison," *arXiv preprint arXiv:1410.7749*, 2014.

[32] I.-A. Császár and R. R. Slavescu, "Building fast and reliable reverse engineering tools with frida and rust," in *2022 IEEE 18th International Conference on Intelligent Computer Communication and Processing (ICCP)*. IEEE, 2022, pp. 289–294.

[33] R. Vallee-Rai and L. J. Hendren, "Jimple: Simplifying java bytecode for analyses and transformations," Technical report, McGill University, Tech. Rep., 1998.

[34] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, "Soot: A java bytecode optimization framework," in *CASCON First Decade High Impact Papers*, 2010, pp. 214–224.

[35] J. Bellizzi, M. Vella, C. Colombo, and J. Hernandez-Castro, "Responding to targeted stealthy attacks on android using timely-captured memory dumps," *IEEE Access*, vol. 10, pp. 35 172–35 218, 2022.

[36] Y. Li, Z. Yang, Y. Guo, and X. Chen, "Droidbot: a lightweight ui-guided test input generator for android," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 2017, pp. 23–26.

[37] S. Berlato and M. Ceccato, "A large-scale study on the adoption of anti-debugging and anti-tampering protections in android apps," *Journal of Information Security and Applications*, vol. 52, p. 102463, 2020.

[38] X. Wang, Y. Zhang, X. Wang, Y. Jia, and L. Xing, "Union under duress: understanding hazards of duplicate resource mismediation in android software supply chain," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3403–3420.

[39] A. Merlo, A. Ruggia, L. Sciolla, and L. Verderame, "You shall not repackage! demystifying anti-repackaging on android," *Computers & Security*, vol. 103, p. 102181, 2021.

[40] "Android Market data, History, Rankings," https://www.androidrank.org/android-most-popular-google-play-apps, [Accessed 21-08-2024].

[41] A. Torres, P. Costa, L. Amaral, J. Pastro, R. Bonifácio, M. d'Amorim, O. Legunsen, E. Bodden, and E. D. Canedo, "Runtime verification of crypto apis: An empirical study," *IEEE Transactions on Software Engineering*, 2023.

[42] "Android Penetration Tools Walkthrough Series: Apktool," https://www.infosecinstitute.com/resources/penetration-testing/

android-penetration-tools-walkthrough-series-apktool/, [Accessed 03-07-2025].